

Shipora Cloud Deploy

Product Requirements Document, Technical Architecture, and Delivery Plan

Version 1.1 | Brand update from Agenlly | August 2026

Executive summary

Shipora turns a GitHub repository into a verified, live cloud application with two user actions: select a repository and click Deploy. Shipora performs the difficult cloud work in the backend: source inspection, deployment planning, secret collection, build execution, database provisioning and wiring, release, health verification, observability, rollback, and usage metering.

The first release targets vibe-coded web applications. Deployments logically belong to a user workspace and project, while Shipora operates the underlying managed cloud infrastructure. The platform later supports user-owned cloud accounts without changing the product workflow.

Product promise

Generate an app. Click Deploy. Get a verified production URL.

Brand system

Asset	Recommended usage
Company/product	Shipora
Primary product	Shipora Deploy
Developer client	Shipora CLI and Shipora for VS Code
Supporting products	Shipora Preview, Shipora Logs, Shipora DB
Tagline	Ship your app, not your DevOps problems.

1. Product definition

Problem

AI-generated code speeds up creation but does not remove production requirements. Users still face build and runtime configuration, port binding, secrets, databases, migrations, service URLs, CORS, domains, logs, scaling, and rollback. A successful build frequently does not mean a working live application.

Target users

- Vibe coders who need to share a real product without learning cloud infrastructure.
- Technical solo founders who want rapid shipping plus logs, rollback, and inspectable state.
- Small product teams that need managed deployment and preview environments without operating DevOps tooling.

Jobs to be done

Situation	Desired outcome
I have code locally or on GitHub	Deploy it without manually configuring cloud resources.
A release fails	Understand the exact blocker and receive a safe next action.
My app needs a database and secrets	Have dependencies provisioned and wired automatically.
I push a change	Choose automatic or manual release without building CI/CD.

Principles

- Two-step default: repository, then Deploy.
- Opinionated automation: support limited stacks exceptionally well.
- Secure by default: no plaintext secrets, silent source edits, or destructive changes.
- Verified success: the release is healthy only after runtime and dependency checks.
- Progressive disclosure: hide cloud detail by default; expose it for advanced users.
- Portability: use a provider-neutral plan internally.

2. PRD and scope

MVP user flow

1. Sign in to Shipora. 2. Connect GitHub and select repository/branch. 3. Shipora scans the pinned commit. 4. User supplies only unavailable external secrets. 5. User clicks Deploy. 6. Backend provisions, builds, releases, wires dependencies, and verifies. 7. Shipora shows a live URL, health, logs, and rollback.

In scope

Area	Release-1 requirement
Input	GitHub App installation; repository and branch selection.
Workloads	Next.js, Vite/React, Node/Express/Bun API, FastAPI, controlled Docker fallback.
Resources	Shipora-managed compute; Neon Postgres; one managed Redis integration when detected.
Operations	Build/runtime logs, event timeline, health checks, rollback to last healthy release.
Environments	Production first; architecture supports preview releases.
Automation	Manual/auto deploy toggle for configured branch.

Out of scope

- Kubernetes, arbitrary Terraform, custom VPCs, GPU jobs, multi-region orchestration, self-managed databases, and arbitrary AWS/GCP accounts.
- Automatic source-code modifications. Shipora may propose a patch but users approve any change.
- Universal framework support. Unsupported repositories enter an explicit advanced/manual path.
- Custom domains, enterprise SSO, and multi-cloud failover before core reliability is proven.

Success metrics

Metric	Initial target
First-deployment verified-success rate	70% or more for supported repositories
Deploy click to verified URL	Median 8 minutes or less
Required configuration inputs	Two or fewer, excluding GitHub authorization
Failure classification coverage	80% or more of failures receive actionable diagnosis
Rollback time	Two minutes or less to a prior healthy release

3. Experience design

Readiness check

Before any billable provisioning, Shipora creates a Deployment Readiness Report. It exposes only blockers and high-confidence warnings, not raw cloud configuration.

```
Repository: ai-resume-builder
Detected: Next.js frontend + FastAPI API + Postgres
✓ Build and service wiring can be configured automatically
Needs from you: OPENAI_API_KEY

[ Deploy to Production ]
```

Visible product screens

Screen	Purpose
Project setup	Repository/branch selection and workspace ownership.
Readiness	Detected services, secret blockers, resource plan, warnings.
Deployment progress	A durable event timeline with safe log tail.
Deployment detail	Live URL, health, commit, services, logs, history, rollback, auto/manual switch.
Advanced	Plan preview, selected overrides, provider/resource metadata.

Error handling

The primary error message must name the failed service and workflow stage, explain the likely cause in plain language, and offer one safe next action. Raw logs remain available but are not the main answer.

```
Example: "The API process started but did not pass its health check. Shipora could not find /health. Select an existing endpoint to check, or add a health route."
```

Secret UX

Shipora scans only secret names and code references; it never imports a local .env file by default. It asks for only external secret values. Platform-generated values—database URL, service URL, JWT key, and CORS origin—are shown as automatically managed.

4. Functional architecture

Lifecycle

DRAFT → ANALYZING → NEEDS_INPUT or PLANNED → PROVISIONING → BUILDING → RELEASING → VERIFYING → HEALTHY. Terminal/side paths: FAILED, ROLLING_BACK, ROLLED_BACK, SUPERSEDED. Each transition emits a persisted event and is idempotent by deployment ID and commit SHA.

Backend workflow

```
GitHub repository → isolated clone → deterministic analyzer → validated DeploymentPlan → user secret input/approval → workflow engine → resource provisioner → isolated build → provider release → bindings → verifier → live URL and metering
```

Repository analyzer

- Inspect package manager lockfiles, Python files, Docker configuration, workspace/monorepo files, migrations, and framework signatures.
- Infer build/start commands only for supported patterns, with confidence and evidence.
- Identify ports, public services, health endpoints, database/queue clients, required environment variable names, and migration commands.
- Use an LLM only to resolve ambiguity after static analysis; validate output against a strict schema and policy engine.

DeploymentPlan example

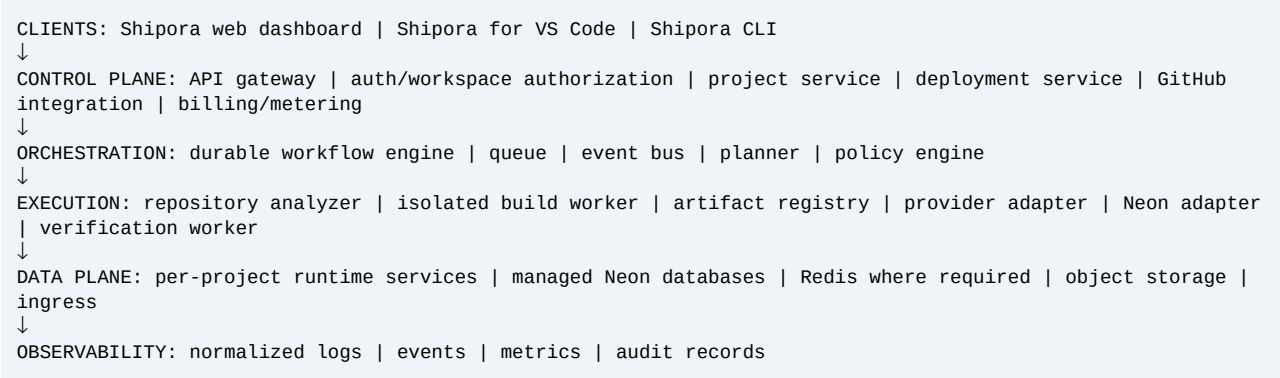
```
{"version":1,"commit":"","services":[{"id":"web","type":"nextjs","public":true},{"id":"api","type":"fastapi","source":"apps/api","healthcheck":"/health"}],"resources":[{"id":"db","type":"neon-postgres"}],"bindings":["api.DATABASE_URL=db.connection_string","web.NEXT_PUBLIC_API_URL=api.public_url"],"requiredSecrets":["OPENAI_API_KEY"]}
```

Service wiring

For recognized full-stack applications, Shipora creates frontend/API bindings automatically: frontend API URL, API allowed CORS origin, database connection string, and private internal service tokens. Users should not copy deployed URLs between dashboards.

5. Technical architecture

Architecture overview



Control-plane components

Component	Responsibility
API gateway	Authentication, authorization, rate limit, request IDs.
Project service	Workspaces, projects, environments, deploy mode.
GitHub integration	App installation, repo selection, signed webhooks, commit metadata.
Analyzer and planner	Static evidence, plan generation, stack-policy validation.
Workflow engine	Durable retries, cancellation, idempotency, cleanup/compensation.
Secrets vault	Encrypted per-project/environment values and access audits.
Provider adapters	Provision, deploy, route, logs, rollback, delete.
Verifier	Health, HTTP, database/dependency, frontend-to-API smoke checks.
Metering	Usage collection and project/organization spend guards.

Multi-tenant model

A user receives a personal workspace by default. Deployments belong to workspace → project → environment → deployment. Shipora initially runs workloads in its managed provider account, but every resource carries `organization_id`, `project_id`, `environment_id`, and `deployment_id`. All reads, writes, secret access, logs, and resource actions enforce the active workspace/project boundary.

6. Data, security, and reliability

Core data model

Entity	Key fields
Organization	id, name, plan, billing account
OrganizationMember	organization_id, user_id, role
Project	id, organization_id, repository, default branch
Environment	id, project_id, deploy mode, target
Deployment	id, environment_id, commit SHA, plan version, state
Service	id, deployment_id, type, URL, health status
ResourceBinding	deployment ID, provider reference, binding name
Secret	project/environment, key, encrypted reference, version
DeploymentEvent	deployment ID, step, level, message, metadata
UsageEvent	organization/project, resource, quantity, time

Security controls

- GitHub App with minimum scope; signed webhook verification and replay protection.
- Envelope encryption for secrets; plaintext never appears in logs, UI after entry, or source control.
- Unprivileged, ephemeral build containers with strict CPU, memory, execution, filesystem, and credential scope.
- Authorization at API level plus tenant filtering/row-level controls in persistence.
- Audit records for deploy, rollback, secret mutation, role changes, and provider connection changes.
- Explicit approval for destructive migrations, deletion, billing-impacting upgrades, and production domain changes.

Reliability controls

- Persist all workflow events before streaming to clients.
- Keep the prior healthy revision live until the candidate release is verified.
- Use retry and compensation steps around provider and provisioning actions.
- Enforce build/runtime quotas, idle cleanup, preview expiration, and spend alerts.
- Require public-route, health endpoint, dependency binding, and basic smoke tests before HEALTHY state.

7. Delivery roadmap

Phase 0 — Foundation (2–3 weeks)

- Choose one deployment provider and define its adapter contract.
- Create 20 representative supported repositories plus intentionally broken fixtures.
- Implement GitHub App, workspace/project/environment model, event schema, audit baseline, and DeploymentPlan schema.

Phase 1 — Verified happy path (4–6 weeks)

- Manual GitHub deployment; analyzer for Next.js, Vite, Express/Bun, FastAPI.
- Required-secret form, encrypted vault, one provider adapter, isolated builds, deployment events, logs, verified URL.
- Managed Neon path and last-healthy rollback.

Phase 2 — Full-stack reliability (4–6 weeks)

- Automatic URL/CORS/database wiring, migration workflow, Redis/worker detection.
- Failure classifier and safe remediation proposals.
- GitHub webhooks, manual/auto deploy switch, preview environments, Neon branch workflow.
- Usage metering, quotas, cleanup, spend safety, and beta support diagnostics.

Phase 3 — Distribution and advanced controls (3–5 weeks)

- Shipora CLI and Shipora for VS Code on the stable deployment API.
- Advanced manifest inspection/export; selected safe overrides.
- Custom domains, team roles, project transfer, and selected BYOC option.

Immediate build order

1. DeploymentPlan and lifecycle state machine. 2. One end-to-end supported repo. 3. Secrets and Neon binding. 4. Verification and rollback. 5. Collect failure data. 6. Expand support based on real failure classes—not feature guesses.

Final product statement

Shipora is the AI deployment engineer for generated applications: it understands a repository, provisions what it needs, deploys it safely, verifies it live, and explains anything that fails.